Behavioral Questions: What Interviewers Really Want

4 min read Updated 24 Mar 2026 Topic 47 of 62

Bookmark Mark done

10.5 - Behavioral Questions: What Interviewers Really Want

🎯 The Most Common Behavioral Questions for React Developers

These aren't React-specific, but they come up in almost every interview alongside technical questions. Having prepared answers shows professionalism.

Q: "Tell me about a challenging bug you fixed."

Framework: Situation, what was broken, how you diagnosed it, what you learned.

> "We had a production issue where user sessions were randomly expiring. I traced it through the network tab and found that our auth token refresh logic had a race condition. Two simultaneous API calls would both detect the expired token and both try to refresh it, but the second refresh would invalidate the first's new token. I fixed it by implementing a token refresh queue: the first call triggers the refresh, and all subsequent calls wait for the same promise. I learned to always think about concurrent edge cases in async flows."

Q: "How do you handle disagreements about technical decisions with your team?"

"I focus on the problem we're trying to solve, not the specific technology. When a colleague wanted to use Redux for a small project and I preferred Zustand, I suggested we list our requirements: we needed global state for a cart and auth, but didn't need middleware or devtools yet. We agreed on Zustand because it matched our actual needs with less boilerplate. If we needed to scale later, migrating would be straightforward. I find that framing decisions around requirements instead of opinions resolves most disagreements."

Q: "How do you stay up to date with React?"

"I follow the React team's blog and Dan Abramov's posts for official updates. I read the React RFC discussions on GitHub to understand what's coming. For practical skills, I build side projects to try new patterns. When a new feature like Server Components ships, I build a small app with it before forming opinions. I also review the code of popular open-source React libraries like Radix UI and React Query to learn from production-grade patterns."

Q: "Describe a time you had to learn something quickly."

"When our team decided to adopt TypeScript, I had a week before starting a new feature. I went through the TypeScript handbook, typed a few existing components to get comfortable with the syntax, and focused on the most common patterns: typing props with interfaces, typing hooks, and handling union types for API responses. I was productive within a few days and became the go-to person for

TypeScript questions on the team within a month. The key was focusing on the 20% of TypeScript that covers 80% of daily use cases."

Q: "What do you do when you're stuck?"

> "First, I spend 15-20 minutes trying to solve it myself: re-read the error, check the docs, simplify the problem. If I'm still stuck, I search for similar issues on Stack Overflow or GitHub Issues. If that doesn't work, I ask a teammate for a quick rubber duck session. I describe the problem clearly, what I've tried, and where I'm blocked. I find that just explaining the problem out loud often reveals the solution. I never spin on a problem for hours without asking for help."

✅ Key Takeaways for Module 10

- **Performance debugging:** Always measure first (Profiler, Lighthouse, bundle analyzer), then fix based on data. Never optimize blindly.
- **Component architecture:** Break the UI into sections, define the tree, decide where state lives, identify communication patterns.
- **State management in large apps:** Think in layers (local, shared UI, global, server state). Match each state type to the right tool.
- **URL to rendered page:** Know the full chain: DNS, TCP/TLS, HTTP, HTML parsing, JS execution, React mounting, Virtual DOM, effects, routing, data fetching, hydration.
- **Behavioral questions:** Prepare structured stories using situation/action/result. Focus on what you learned, not just what you did.

📚 Module 10 Complete!

#	Question Type	What it Tests
10.1	"This app is slow"	Performance diagnosis, profiling, systematic fixing
10.2	"Design the architecture"	Component decomposition, state placement, data flow
10.3	"Manage state at scale"	State layering, tool selection, server vs client state
10.4	"URL to rendered page"	Full-stack knowledge: DNS, HTTP, React rendering, hydration
10.5	Behavioral questions	Communication, teamwork, learning, problem-solving

QUICK CHECK — no marks, just for you.

1. In the sample answer to "Tell me about a challenging bug you fixed," sessions were randomly expiring because two simultaneous API calls both detected the expired token and both tried to refresh it. According to the lesson, how was this fixed?

- ○ A token refresh queue: the first call triggers the refresh and all subsequent calls wait for the same promise
- ○ Retrying the failed request with exponential backoff until a valid token was returned
- ○ Increasing the token expiry window so two calls could never race within the same window
- ○ Storing the refreshed token in localStorage so both calls could read the same value

Check answer

resolves most such disagreements?

- Framing the decision around the project's actual requirements rather than around opinions
- Deferring to whichever teammate has the most experience with the technology in question
- Prototyping both options and letting benchmark numbers decide the winner
- Choosing the library with the larger ecosystem so the team is never blocked later

Check answer

3. Based on the lesson's answer to "What do you do when you're stuck?", which sequence best matches the escalation ladder it describes?

- Search Stack Overflow first, then self-debug for an hour, then escalate to a senior engineer
- Keep working the problem alone until it is solved, escalating only if it blocks a release
- Self-debug for 15-20 minutes, then search Stack Overflow or GitHub Issues, then ask a teammate for a rubber duck session
- Ask a teammate immediately so no time is wasted, then search issues, then read the docs yourself

Check answer

Previous
Scenario: "Walk Me Through What Happens When You Type a URL and See a React

Next
Machine Coding Round: Strategy & Setup

LEARN MORE

ARTICLES & DOCS

- The STAR method (overview)
  wikipedia.org

VIDEOS



- SWE behavioral questions (STAR answers)
  YouTube



- STAR method for developers
  YouTube



- STAR method with examples
  YouTube

Hand-picked references for this lesson.